Grokfast Project Brief

CSCI 567 Final Project · Vishal Patel, Salman Sattar, Tomohiko Uemura, Chinenye Okoro

Lee et al., *Grokfast: Accelerated Grokking by Amplifying Slow Gradients*, arXiv:2405.20233

CENTRAL CLAIM

Amplifying the slowly varying part of each parameter's gradient makes models that grok generalize far sooner (about 50× fewer steps on modular arithmetic), because slow gradient components drive generalization while fast components drive overfitting.

Method: before each optimizer step, add λ × a low-pass filtered gradient history to the current gradient. `ema = α·ema + (1-α)·g; g ← g + λ·ema` (Grokfast-EMA), or a 100-step moving average (Grokfast-MA). Two lines in the training loop.

KEY EXPERIMENTS IN THE PAPER

Task (small data)	Model	Reported effect
x·y mod 97, 50% train	2-layer Transformer	Steps to 95% val: `39,890` → `790` (MA, 50×), `910` (EMA, 44×)
MNIST, 1,000 images, 8× init	3-layer MLP	Steps to threshold `44,022` → `2,001` (22×)
QM9, 100 molecules, 3× init	Graph CNN	Min val MSE `0.00659` → `0.00348`
IMDb, 1,000 reviews	2-layer LSTM	Best val acc `84%` → `90%`

Ablations

- Window and gain sweeps have an interior optimum; stronger filtering is not always better.
- Using only the averaged gradient (dropping the current one) is slower and unstable.
- Starting the filter after 500 steps still reaches 95% val in `1,920` steps.
- Weight decay interacts strongly with the filter.

What we'd flag

- The 50× combines two changes. The filter alone gives 13.6× (2,940 steps); adding weight decay 0.01 brings it to 50×. The baseline uses no weight decay.
- The proof that gradient filtering equals update filtering assumes a linear optimizer (SGD, momentum), but the experiments use Adam.
- One validation split serves both tuning and reporting; no seed variance for the speedups.
- MNIST threshold is reported as 95% in a table but drawn at 85% in the figure.

CODE: LOCATED, SET UP, AND RUN

Team repo vip11017/csci567-grokfast, forked from the official code (ironjr/grokfast), with Apple-silicon (MPS) support added. Steps to 95% val on modular arithmetic, one seed each:

Run	Ours	Paper
Baseline (wd 0)	`58,600`	`39,890`
Grokfast-MA (w 100, λ 5, wd 0.01)	`790`	`790`
Grokfast-EMA (α 0.98, λ 2, wd 0.005)	`910`	`910`

- The filtered runs match exactly. Our baseline was 1.5× slower, so seed variance alone moves the "50×" ratio a lot.
- A re-run of Grokfast-MA on a second machine (Windows laptop, CPU only, ~6 min for 1,000 steps) hit 95% val at step `790` again.
- The script's "weight decay" goes into `torch.optim.Adam`, so it is coupled L2 regularization, not decoupled AdamW decay.

DRAFT ALTERNATIVE HYPOTHESIS

> **Regularization, not gradient frequency, drives most of the speedup.** The filter changes how fitting trades off against weight size; a well-tuned baseline with weight decay alone may close most of the gap.
>
> **Prediction:** with matched optimizer, learning-rate search and seeds, a tuned no-filter baseline (Adam-L2 and AdamW, decay up to 1.0) reaches 95% val in far fewer than 40k steps.
>
> **Test:** filter on/off × weight decay {0, 0.001, 0.005, 0.01, 0.05, 0.1, 1.0}, ≥3 seeds, logging parameter and update norms. Report median steps to 95% on a held-out test split.

Also drafted. H1: it's ordinary momentum and step size (compare against tuned Adam β_1 and momentum SGD at matched update norms). H3: the gains depend on induced grokking (MNIST across train sizes 1k–50k and init scales 1–8).

Mechanism check: a high-pass control (`g + λ(g - ema)`) should not help if slow components are what matter.

QUESTIONS FOR THE TAS

1. Is replication plus one well-controlled hypothesis (H2) enough scope, or should we also cover H1 or H3?
2. Compute: each baseline run takes ~40–60k steps, and we want ≥3 seeds per setting. Is there GPU access we should use (e.g. CARC)?
3. Does the high-pass control seem like a fair test of the paper's mechanism?